

Day 1 – Basics of Linked List

ITSRUNTYM

1. Introduction & Need for Linked List

What is a Linked List?

A **Linked List** is a collection of **nodes**, where:

- Each node stores **data** and the **address of the next node**.
- The **first node** is called the **head**.
- The last node's next pointer is null (in singly linked lists).

Why not just use Arrays?

Array

- Fixed size (once declared, can't grow/shrink).
- Stored in **contiguous** memory.
- Inserting/deleting at the middle requires **shifting** elements → **O(n)**.

Linked List

- **Dynamic size** — grows/shrinks as needed.
- Elements can be scattered anywhere in memory.
- No shifting needed — just change pointers.
- But uses **extra memory** for storing pointers.

Real-World Analogy

Think of a **train**:

- Each coach = a **node**.
- The coupler = the **pointer/reference** to the next coach.
- You can easily attach/detach coaches without shifting all others.

Diagram

[Data | Next] -> [Data | Next] -> [Data | Next] -> null

Example:

[10 | next] -> [20 | next] -> [30 | null]

2. Structure of a Node

A **node** in Java looks like this:

```
class Node {
```

```

int data; // Value stored
Node next; // Address of next node
Node(int data) {
    this.data = data;
    this.next = null;
}
}

```

- data → stores the value (like 10, 20).
- next → stores the reference to the next node.
- If it's the last node → next = null.

Example in Memory:

Address	Data	Next (address)
100	10	104
104	20	108
108	30	null

3. Types of Linked Lists

1) Singly Linked List

- Each node points to **next node only**.
- Traversal is only **forward**.

Head -> [10|next] -> [20|next] -> [30|null]

2) Doubly Linked List

- Each node has **prev** and **next** pointers.
- Can traverse **both forward and backward**.

null <- [10|prev,next] <-> [20|prev,next] <-> [30|prev,null]

3) Circular Linked List

- Last node points back to **head**.
- No null in next.

[10|next] -> [20|next] -> [30|next] -> back to 10

4. Basic Operations (Today's Focus)

We'll only do **Head & Tail** insertions/deletions + Traversal.

A) Creating a Linked List Manually

```
public class LinkedListDemo {
    Node head;
    static class Node {
        int data;
        Node next;
        Node(int data) { this.data = data; }
    }
    public static void main(String[] args) {
        LinkedListDemo list = new LinkedListDemo();

        // Create nodes
        list.head = new Node(10);
        Node second = new Node(20);
        Node third = new Node(30);

        // Link them
        list.head.next = second;
        second.next = third;

        // Print list
        list.printList();
    }
    void printList() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }
}
```

Output:

10 -> 20 -> 30 -> null

B) Traversing a Linked List

- Start from head

- Follow next pointers until null
- Time Complexity → **O(n)**

```
void printList() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }
    System.out.println("null");
}
```

C) Insertion at Beginning

1. Create new node.
2. Point its next to current head.
3. Make head point to new node.

```
void insertAtHead(int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
}
```

D) Insertion at End

1. Create new node.
2. Traverse to last node.
3. Set last node's next to new node.

```
void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) { head = newNode; return; }
    Node temp = head;
    while (temp.next != null) temp = temp.next;
    temp.next = newNode;
}
```

Insert at a specific position (0-based index)

```
public void insertAtPosition(int index, int data) {
    if (index < 0) {
```

```

        System.out.println("Invalid index");
        return;
    }
    if (index == 0) {
        insertAtHead(data);
        return;
    }

    Node temp = head;
    for (int i = 0; i < index - 1 && temp != null; i++) {
        temp = temp.next;
    }

    if (temp == null) {
        System.out.println("Index out of bounds");
        return;
    }

    Node newNode = new Node(data);
    newNode.next = temp.next;
    temp.next = newNode;
}

```

E) Deletion from Beginning

- Move head to head.next

```

void deleteAtHead() {
    if (head != null) head = head.next;
}

```

F) Deletion from End

1. Traverse to 2nd last node.
2. Set its next to null.

```

void deleteAtEnd() {
    if (head == null) return;
    if (head.next == null) { head = null; return; }
    Node temp = head;
    while (temp.next.next != null) temp = temp.next;
    temp.next = null;
}

```

Delete at a specific position (0-based index)

```
public void deleteAtPosition(int index) {
    if (index < 0 || head == null) {
        System.out.println("Invalid index or empty list");
        return;
    }
    if (index == 0) {
        deleteAtHead();
        return;
    }

    Node temp = head;
    for (int i = 0; i < index - 1 && temp != null; i++) {
        temp = temp.next;
    }

    if (temp == null || temp.next == null) {
        System.out.println("Index out of bounds");
        return;
    }

    temp.next = temp.next.next;
}
```

Search Operation

```
public boolean search(int key) {
    Node temp = head;
    while (temp != null) {
        if (temp.data == key) return true;
        temp = temp.next;
    }
    return false;
}
```

Print the list

```
public void printList() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }
}
```

```

    }
    System.out.println("null");
}

```

5. Complexity Table

Operation	Time	Space
Traversal	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(1)$
Insert at end	$O(n)$	$O(1)$
Delete at head	$O(1)$	$O(1)$
Delete at end	$O(n)$	$O(1)$

Key Changes from Singly to Doubly Linked List

1. Node Structure

```

static class Node {
    int data;
    Node next; // points to next node
    Node prev; // points to previous node
    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}

```

2. Insert at Head

- In a singly linked list: only next is updated.
- In a doubly linked list: also update prev of old head.

```

public void insertAtHead(int data) {
    Node newNode = new Node(data);
    if (head != null) {
        newNode.next = head;
        head.prev = newNode; // extra step for doubly LL
    }
    head = newNode;
}

```

3. Insert at End

- Update both next of last node and prev of new node.

```
public void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        return;
    }
    Node temp = head;
    while (temp.next != null) temp = temp.next;
    temp.next = newNode;
    newNode.prev = temp; // link back
}
```

4. Insert at Position

- Update **4 pointers**: prev and next of new node, prev of next node, next of previous node.

```
public void insertAtPosition(int index, int data) {
    if (index < 0) {
        System.out.println("Invalid index");
        return;
    }
    if (index == 0) {
        insertAtHead(data);
        return;
    }
    Node temp = head;
    for (int i = 0; i < index - 1 && temp != null; i++) {
        temp = temp.next;
    }
    if (temp == null) {
        System.out.println("Index out of bounds");
        return;
    }
    Node newNode = new Node(data);
    newNode.next = temp.next;
    newNode.prev = temp;

    if (temp.next != null) {
        temp.next.prev = newNode; // link back from next node
    }
    temp.next = newNode;
}
```

```
}
```

5. Delete at Head

- In addition to moving head, clear prev of the new head.

```
public void deleteAtHead() {  
    if (head != null) {  
        head = head.next;  
        if (head != null) head.prev = null;  
    }  
}
```

6. Delete at End

- Move to last node, update next of second-last node, and clear prev of last node.

```
public void deleteAtEnd() {  
    if (head == null) return;  
    if (head.next == null) {  
        head = null;  
        return;  
    }  
    Node temp = head;  
    while (temp.next != null) temp = temp.next;  
    temp.prev.next = null; // unlink last node  
}
```

7. Delete at Position

- Update **both** next and prev pointers of surrounding nodes.

```
public void deleteAtPosition(int index) {  
    if (index < 0 || head == null) {  
        System.out.println("Invalid index or empty list");  
        return;  
    }  
    if (index == 0) {  
        deleteAtHead();  
        return;  
    }  
    Node temp = head;  
    for (int i = 0; i < index && temp != null; i++) {
```

```

        temp = temp.next;
    }
    if (temp == null) {
        System.out.println("Index out of bounds");
        return;
    }
    if (temp.prev != null) temp.prev.next = temp.next;
    if (temp.next != null) temp.next.prev = temp.prev;
}

```

8. Search

- Same as singly linked list (no pointer change needed).

9. Traversal (Forward & Backward)

```

public void printForward() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " <-> ");
        temp = temp.next;
    }
    System.out.println("null");
}
public void printBackward() {
    if (head == null) return;
    Node temp = head;
    while (temp.next != null) temp = temp.next; // go to tail
    while (temp != null) {
        System.out.print(temp.data + " <-> ");
        temp = temp.prev;
    }
    System.out.println("null");
}

```

Main Changes Summary Table

Operation	Extra Step in Doubly Linked List
Insert Head	Update prev of old head
Insert End	Update prev of new node
Insert Pos	Update prev of next node
Delete Head	Update prev of new head
Delete End	Use prev to unlink last node
Delete Pos	Update both next & prev

6. Practice for Students

1. Create a linked list with 5 nodes and print it.
2. Insert 2 elements at head, then print.
3. Insert 2 elements at end, then print.
4. Delete first and last node, then print.
5. Count total nodes in the list.